Evidence Balanced and Explainable Indian Legal Search

CSD358 Information Retrieval · Track T6: Vertical Search for Law · 7 October 2026

Project scope: search over English Supreme Court of India judgments dated 2018–2024. Team member names and a verified contribution record were not present in the repository; see Section 7 and the submission checklist.

1. Problem and Track Relevance

The project addresses a practical legal research problem: finding passages in judgments that relate to a legal proposition and making the evidence easier to inspect. A student, lawyer, or other legal researcher may need to locate judgments about a question such as whether a dying declaration alone can support a conviction. Searching only for matching words can return passages that discuss the same topic but do not answer the question, and a ranked list alone does not explain why one passage appeared above another.

The system indexes Supreme Court of India judgments from 2018 through 2024. Its intended users are people exploring that bounded collection, not people seeking a complete statement of law. It returns passages linked to their original judgment PDFs, with case metadata and a ranking trace. This matters because legal documents are long and structured: a concise headnote, title, cited statute, date, and bench information can help a researcher inspect a result beyond its body text.

This fits Track T6, vertical search for law. The assignment asks for a search engine that uses document structure such as zones, metadata, citations, and authority. The implementation uses headnote/body zones, title and statute signals, year and bench metadata, and bench size as a limited static authority proxy. It does not implement citation-graph authority or treatment status such as “overruled.”

The project materials refer to four related research efforts, consulted at abstract or summary level rather than as full-paper reproductions. Malik et al. (2021) introduced ILDC for Indian judgment prediction and explanation; it provides context for Indian legal corpora, but this project uses a different AWS-hosted judgment collection and a retrieval task. Joshi et al. (2023), U-CREAT, introduced the IL-PCR prior-case retrieval benchmark and studied event-based case retrieval; it motivates the difficulty of legal relevance, while this project uses a classical lecture-based passage retriever. Paul et al. (2025), IL-PCSR, combines precedent and statute retrieval in one corpus; it highlights a possible extension, since the current project only indexes judgments and extracts statute mentions. Vishnuprabha et al. (2025), AQgR, explores question-guided case retrieval with structured summaries and LLMs; it is related to the project’s optional response layer, but the current system does not generate query expansions or retrieve with embeddings. These works informed the problem framing, not the code or evaluation.

2. How We Used Information Retrieval

The main system is a hand-built classical retrieval pipeline. The index is built once from PDF text and metadata. At query time, retrieval generates candidate passages, scores them, selects top results, aggregates passages to judgments, and displays an explanation of the score. A separate stance stage may classify the retrieved passages; it does not change the topical retrieval score.

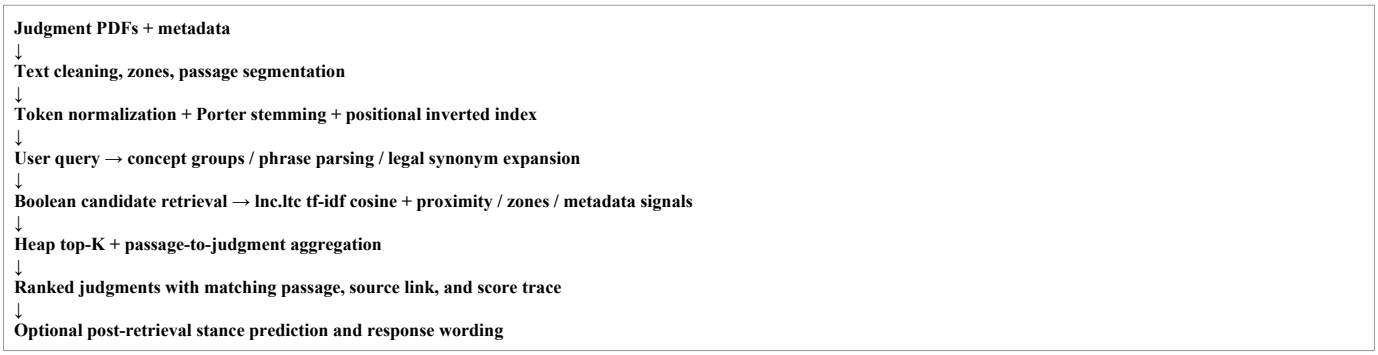


Figure 1. Information retrieval pipeline implemented in the project. The stance and response stages are downstream of retrieval and do not alter the topical score.

IR techniques and where they appear

Technique	Implementation and effect	Why / code location
Structured documents and passages	Judgment PDFs are cleaned, split into headnote/body zones, and segmented into paragraph-based passages; judgments are the display unit.	Long judgments make passage retrieval useful; zones preserve context. irlaw/build_index.py
Normalization and positions	Case folding, tokenization, a light stop list, Porter stemming; token offsets are retained in postings.	Normalize variants while preserving positions for exact/legal phrase and proximity matching. irlaw/textproc.py; build_index.py
Positional inverted index	Term dictionary and compressed arrays for passage postings, term frequency and token positions; project reports about 29.7M stored positions.	Fast lookup plus phrase and proximity checks without scanning every document. Index in irlaw/engine.py
Boolean and query processing	Concept groups combine a term with curated legal synonyms; groups are ANDed and variants ORed. Common groups/variants are excluded from filtering; intersections start with rare groups and relax when needed.	Balances precision and candidate availability for legal vocabulary variation. parse_query(), search() in irlaw/engine.py
Phrase and proximity matching	Phrase parsing covers statute names/numbers and selected legal phrases; minimal windows measure how close query concepts occur.	Phrase matches protect legal terms of art; proximity favors passages where concepts form one point. parse_query(), smallest_window(), search()
Vector space / TF-IDF	SMART lnc.ltc: log tf and cosine-normalized passage weights; query uses log tf × idf; synonym variants are down-weighted.	A standard lexical ranking baseline with length normalization. query_vector(), search()
Top-K and aggregation	heapq selects a bounded set of passages; high-ranked passages are retained as champions for stance cues; passage scores are aggregated to judgments.	Keeps positional rescoring bounded and presents cases rather than duplicate fragments. search(), irlaw/engine.py
Zones, fields, static quality	Headnote boost; title concept hits; statute field; year/court/disposal/bench metadata; $g(d)=\log_2(\text{bench size})$. Net score adds these to the topical and proximity signals.	Uses legal structure and available metadata. Bench size is only a proxy, not citation authority. search() trace in irlaw/engine.py; fields in build_index.py
Explainable scoring	Each hit carries TF, DF/IDF, weights, proximity window, phrase/zone/field inputs and a summed score trace; a verifier recomputes the trace from source text.	Makes ranking inspectable and supports debugging. irlaw/faithfulness.py; evalkit/verify_faithfulness.py

Corpus size reported by the project: 5,776 judgments, 195,640 passages, and 139,115 indexed terms. The corpus is Supreme Court judgments from 2018–2024; one of 5,777 PDFs was dropped after extraction failure or insufficient text. The data is sourced from the AWS Open Data Indian Supreme Court Judgments collection, which packages judgment PDFs from the court/eCourts source.

For a query, the parser creates concept groups, expands selected legal synonyms, and applies Boolean candidate generation using postings. The candidate passages receive lnc.ltc cosine scores. The engine then considers positional proximity, headnote weighting, stance phrase signals in its champion list, title and statute matches, and bench-size g(d), before returning the top judgments and passage score traces. The UI can also show the plain tf-idf baseline. In the current app, a separate classifier runs after topical retrieval; if enabled, its prediction is shown alongside the unchanged retrieval evidence.

Version distinction: stance-phrase cues and the champion-list mechanism describe the earlier phrase-cue stance experiment that remains available for evaluation. The current app retrieves topical candidates first and then applies a local rule-based or explicitly selected language-model classifier. That classifier does not alter retrieval ranking. Metrics for the phrase-cue experiment must not be presented as accuracy for the current classifier.

3. Beyond IR

Component	What we used and why	How it supports IR
PDF/data preparation	PyMuPDF extracts text; pandas/pyarrow read metadata; NumPy stores index arrays.	Converts the source corpus into searchable text and fields.
Stance classification	Rule-based local classifier is the no-key default; optional model classifier can be selected. It predicts relevance, speaker role, stance, evidence span, and confidence.	Organizes retrieved evidence by apparent position; it is post-retrieval and does not change ranking. Predictions remain unverified.
Response generation	Regular/Professional response wording from a local Ollama model, optional Anthropic API, or deterministic template. Quotes and citations are checked against the frozen evidence pack.	Makes results easier to read while limiting the response to retrieved items; it does not retrieve or rerank.
Streamlit interface	Search controls, categorized results, metadata, source links, score trace, and expandable IR intermediates.	Lets users inspect retrieval results and evidence; it is the interface, not an IR contribution.
Evaluation utilities	Pooling, labeling helpers, metrics, ablations, and score-trace verification.	Measures retrieval and stance separately and checks that explanations match the scoring code.

4. Novelty and Creativity

Required by the project track: build a domain search engine that uses legal document structure. The assignment's sample ideas include legal precedent retrieval with tf-idf and authority, as well as zone and parametric search.

Standard implementation choices: a positional inverted index, tf-idf cosine ranking, Boolean retrieval, stemming, passage segmentation, metadata fields, and a Streamlit UI are established IR engineering methods. Their implementation is useful but should not be presented as novel by themselves.

The project's additional contribution is the combination of a structured legal passage index, explicit stance-related phrase queries and proximity signals in the measured retrieval pipeline, a per-signal score trace verified against source text, and a frozen evidence pack used to check that Regular and Professional wording refer to the same retrieved evidence. The newer classifier adds role-aware post-retrieval labels and can abstain, but its measured performance is not strong enough to claim accurate stance understanding. These features improve inspectability and let the team quantify failure modes; they do not establish a new state of the art.

The results support a modest contribution: structured Boolean concepts and zones improve topical retrieval against the simple cosine baseline on this pilot set. The stance-sorted output remains a weak point. A realistic next novelty step would be to add verified citation links or treatment signals, or to compare a carefully evaluated semantic reranker against the current lexical engine. Either would need a suitable benchmark and independently checked labels.

5. Evaluation

A. Query set and intent

The evaluation uses 23 proposition queries: 3 DEV queries used to select weights, 12 TEST queries, and 8 FRESH queries written after an earlier system freeze. FRESH is the cleanest split. The queries cover criminal evidence and procedure, employment, bail, property, maintenance, and compensation. Six representative examples are shown below; each asks for cases or passages addressing the named legal proposition.

Query	User intent
Can a dying declaration alone form the basis of conviction?	Find decisions on whether this evidence can independently sustain conviction.
Is a confession made to a police officer admissible in evidence?	Find the admissibility rule and relevant exceptions or limits.
Can a High Court quash an FIR under Section 482 CrPC?	Find judgments on the scope of inherent powers to quash an FIR.
Is prior sanction required to prosecute a public servant for corruption?	Find decisions on when sanction is a prerequisite to prosecution.
Can a person claim title by adverse possession against the true owner?	Find judgments addressing the legal requirements and limits of the claim.
Is a divorced wife entitled to maintenance under Section 125 CrPC?	Find decisions about eligibility and qualifications for maintenance.

B. Relevance judgements and metrics

The evaluation pools top results from multiple system variants and labels passages using N (not relevant), T (topically relevant but no stance), S (supports), O (opposes or limits), and M (mixed). For topical relevance, T/S/O/M count as relevant and N does not. A judgment is evaluated using its best retrieved passage. Unjudged documents are treated as non-relevant, so recall is pooled recall over this incomplete pool, not corpus-wide recall. In total, the project report records 771 judged passages: 98 S, 108 O, 69 M, 333 T, and 163 N. Claude assisted in assigning these labels; the team has not independently re-judged them and no inter-annotator agreement is available.

P@k is the proportion of the first k results judged relevant. Recall is the share of relevant items found among those judged in the pooled set. Support/opposition list precision requires the stance label to match exactly. "Both-sides coverage" is the fraction of queries with at least one correctly judged S and one O in the top ten. MAP is not reported in the saved summary results. No significance test was run; the query set is small.

C. Baseline and results

System (20 held-out queries)	P@5	P@10	Pooled recall S	Pooled recall O	Queries with S and O in top 10
A. lnc.ltc tf-idf cosine baseline	0.84	0.78	0.38	0.35	0.50
B. + Boolean concepts and synonyms	0.92	0.89	0.42	0.33	0.40
C. + proximity	0.91	0.89	0.37	0.36	0.50
D. + zones and statute field	0.91	0.88	0.42	0.48	0.60
E. + bench-size authority proxy	0.91	0.89	0.42	0.48	0.60

Baseline comparison: the full E pipeline improves P@10 from 0.78 to 0.89 on TEST+FRESH (20 held-out queries). On FRESH alone, P@10 rises from 0.79 to 0.90; support recall rises 0.33 to 0.65, opposition recall 0.42 to 0.51, and both-sides coverage 0.50 to 0.62. These are macro averages from the saved pilot evaluation, not significance-tested results. Boolean concepts/synonyms produce the largest P@10 gain; adding zones/statute fields raises opposing-passage recall and both-sides coverage. The bench-size authority signal changes no measurable metric in this ablation.

The old phrase-cue support/opposition lists have strict precision 0.20/0.22 on the 20 held-out queries, and both lists contain a correctly labeled item for 40% of queries. Thus better topical ranking does not imply reliable stance assignment.

D. Newer stance classifier evaluation

The local rule-based classifier was compared with the AI-assisted, unreviewed reference on 440 held-out passages: support precision 8/27 = 0.30, opposition precision 10/29 = 0.34, but it made a decided stance prediction on only 56/440 passages (13%). The optional Ollama qwen2.5:7b classifier was compared on the same type of unverified reference: for TEST it achieved support precision 0.48 (42 predictions) and opposition precision 0.27 (11 predictions) at 22% decided coverage; for FRESH it achieved support precision 0.30 (33 predictions), opposition precision 0.00 (3 predictions), and 23% decided coverage. It predicted MIXED zero times. These are agreement-with-reference measurements, not trustworthy accuracy estimates. The reference is itself AI-assisted and unreviewed. The Claude/API model path has no reported accuracy evaluation. Quote checks, citation checks, and offline fake-client tests check output format and grounding constraints; they do not establish stance correctness.

E. What remains for a stronger evaluation

The current stored result files provide aggregate metrics by split and system stage, but not a submission-ready per-query comparison table with independently verified judgments. Before making stronger claims, the team should independently judge a random sample or, preferably, the full pooled set; record disagreements and agreement; recompute topical and stance metrics; and report per-query results alongside macro averages. Add MAP only if relevance labels and ranked judged lists are complete enough to define it. A graph should be generated only from the verified metric table; the report therefore uses numerical tables and does not draw a graph from unverified labels.

6. Limitations and Next Steps

• Coverage: the index contains only English Supreme Court judgments from 2018–2024; it excludes High Court decisions and statutes as separate documents. A relevant authority outside this collection cannot be retrieved.

• Text and metadata: PDF extraction, passage boundaries, and automatic headnote/statute parsing may be imperfect. One PDF was excluded after text extraction checks. A metadata citation discrepancy is documented in the existing report.

• Query understanding: the engine relies on lexical terms, curated synonyms, Boolean groups, and proximity. It can miss a legal concept expressed in different words or retrieve similar wording that does not answer the issue.

• Ranking: bench size is only a limited authority proxy; citation counts and negative treatment are not indexed. The measured authority feature did not improve the pilot metrics.

• Stance: both rule and LLM predictions remain imperfect; sparse opposing predictions and zero MIXED predictions in the reported Ollama run are warning signs. Automatic output must not be presented as verified legal interpretation.

• Evaluation: 23 queries and 771 pooled labels are a small pilot. Labels are AI-assisted, not independently validated; pooled recall is incomplete; significance tests and agreement statistics are absent. Response summaries were not evaluated for faithfulness beyond exact-quote and citation checks.

Roadmap

Phase	Next work
1. Evaluation validity	Independently judge the pooled passages; report annotation agreement and per-query errors.
2. Retrieval	Re-tune score weights on a dev set for the current tf-idf pipeline; inspect vocabulary-mismatch failures and evaluate phrase/zone changes separately.
3. Stance	Evaluate local and model classifiers on a verified held-out set; report precision, recall, coverage, and confusion by speaker role; preserve abstention.
4. Legal structure	Add citation links and treatment/authority signals only after reliable extraction; consider statutes or High Court documents as a separate indexed collection.
5. User evaluation	Ask law students to complete a small set of search tasks and record whether the passages help them find the relevant legal principle.

7. Work Division

The repository contains a planned four-role division, but it does not identify the actual students or verify which person completed each task. [MISSING — NEED TO ADD: team member names and confirmed work division.] The table below preserves the recorded role areas without inventing names or asserting equal contributions. Replace the member labels and confirm the work performed before submission.

Team member (name missing)	Recorded work area; verify attribution
Member 1 — [MISSING: name]	Data/source research, extraction, passage zones, preprocessing and index.
Member 2 — [MISSING: name]	Retrieval, ranking, Boolean concepts, proximity, and stance-query components.
Member 3 — [MISSING: name]	Score explanation, faithfulness checks, judging protocol, metrics and ablations.
Member 4 — [MISSING: name]	Response layer, Streamlit interface, integration, documentation and video.

AI Use Declaration

AI tools were used substantially. The project repository states that Claude Code generated most of the code and project documentation, helped with debugging and design, and assisted in creating the pooled relevance/stance judgments and evaluation workflow. The judgment labels used for the reported metrics are therefore AI-assisted and have not been independently verified by the student team. Optional LLM components generate stance predictions and response wording at runtime; local Ollama was used in recorded summary/classifier experiments. Codex was used to inspect the project materials and prepare this final report. These components are not ground truth. The team remains responsible for checking the implementation, metrics, citations, and report before submission. The exact division between AI-generated and student-authored code should be confirmed by the team if the course requires a more detailed disclosure.

References

• Malik, V., Sanjay, R., Nigam, S. K., Ghosh, K., Guha, S. K., Bhattacharyya, A., & Modi, A. (2021). ILDC for CJPE: Indian Legal Documents Corpus for Court Judgment Prediction and Explanation. Proceedings of ACL-IJCNLP 2021, 4046–4062. https://aclanthology.org/2021.acl-long.313/ (related work; consulted at abstract/summary level).

• Joshi, A., Sharma, A., Tanikella, S. K., & Modi, A. (2023). U-CREAT: Unsupervised Case Retrieval using Events Extraction. Proceedings of ACL 2023, 13899–13915. https://aclanthology.org/2023.acl-long.777/ (related work; consulted at abstract/summary level).

• Paul, S., Goyal, P., Ghumare, D., Ghosh, S., & Modi, A. (2025). IL-PCSR: Legal Corpus for Prior Case and Statute Retrieval. Proceedings of EMNLP 2025. https://aclanthology.org/2025.emnlp-main.738/ (related work; consulted at abstract/summary level).

• Vishnuprabha, V., Viswanathan, D. M., Rajesh, R., & Pillai, A. V. (2025). Augmented Question-guided Retrieval (AQgR) of Indian Case Law with LLM, RAG, and Structured Summaries. arXiv:2508.04710. https://arxiv.org/abs/2508.04710 (related work; consulted at abstract/summary level).

• Dattam Labs. Indian Supreme Court Judgments [dataset]. AWS Open Data Registry. https://registry.opendata.aws/indian-supreme-court-judgments/ (accessed 7 October 2026; CC BY 4.0 as stated in project materials).

• Project software/library references: NLTK Porter stemmer, PyMuPDF, NumPy, pandas, PyArrow, Streamlit, and Ollama are used as implementation tools. The project authors' retrieval/scoring logic is implemented in the repository files cited in Section 2.

FINAL THINGS WE NEED TO DO BEFORE SUBMISSION

• Add all team members' actual names and confirm the contribution table against work completed.

• Have the team independently verify relevance/stance labels (at least a random sample) and report agreement; then recalculate the headline metrics if labels change.

• Confirm the exact corpus counts, evaluation file versions, and cited paper/dataset details against the submitted code and course materials.

• Decide which stance configuration the recorded demo shows; label its outputs as automatic predictions and do not claim verified accuracy.

• Check the final report page count against the instructor's limit and add repository/video links if required.